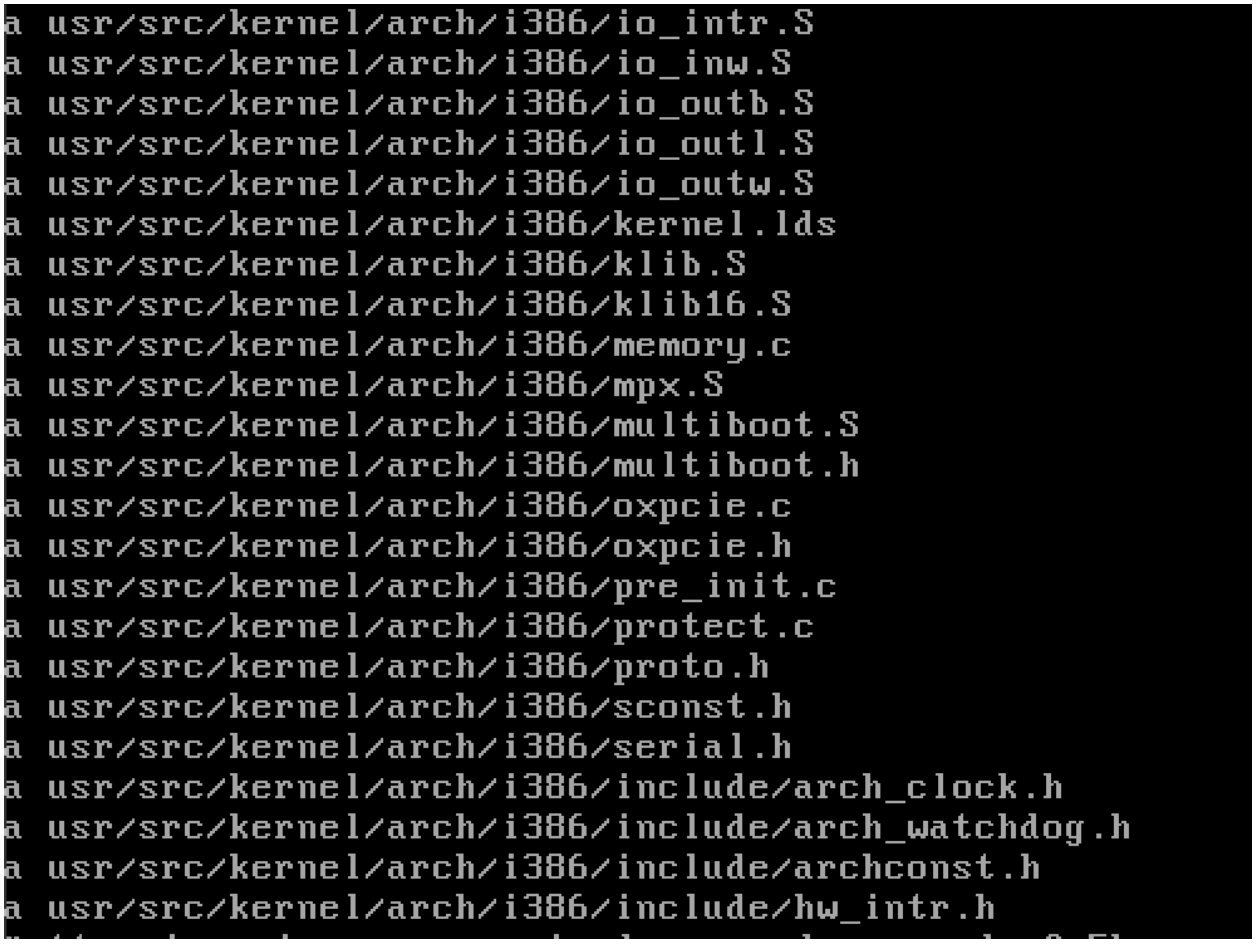
Task 1: Build a modified kernel

Step 1: Start MINIX and back up the kernel

Approach:
I started MINIX in QEMU and logged in as root. Before making any kernel changes, I created a backup of the kernel source using tar:

tar cvf safe.tar /usr/src/kernel

This archived the kernel source so I would have a copy to return to if my changes broke the system.



I also referred to the usage, monitor, boot, shutdown, and tar documentation while doing the lab.

Problems Encountered:
N/A.

Solution:
N/A.

Lessons Learned:
I learned that tar is used for backups, usage explains MINIX configuration and make hdboot, monitor explains the boot monitor, boot explains the startup process, and shutdown explains how to safely shut down or reboot the system.

Step 2: Locate the idle code

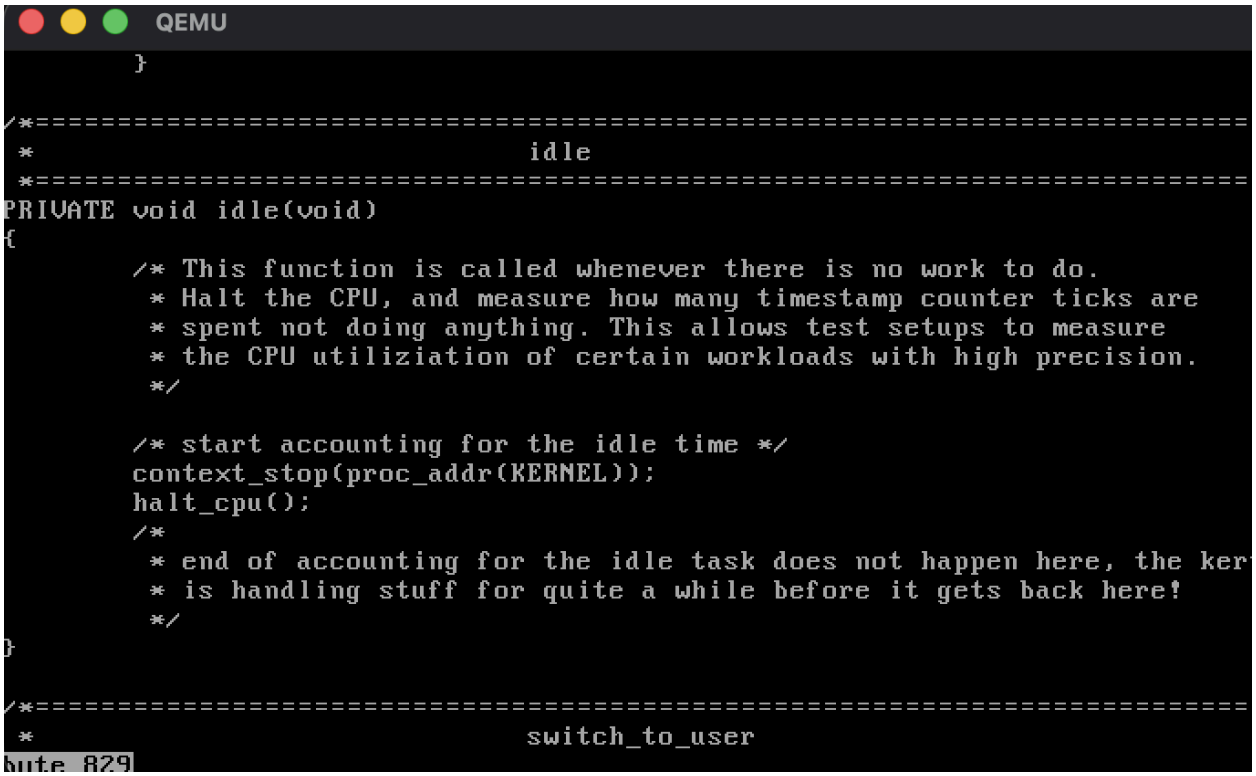
Approach:
I needed to modify the kernel so that an @ appears when the system is idle. Since the kernel source is in /usr/src, I went into the kernel directory and searched for idle-related code:

cd /usr/src/kernel
*grep -n "idle" *.c*

This showed that idle() was defined in proc.c, so I viewed that section of the file:

sed -n '80,145p' proc.c | more

The comment inside idle() said the function is called whenever there is no work to do, so I knew this was the right location for the modification.



Problems Encountered:
The QEMU screen was small, so it was hard to read long command output.

Solution:
I used *more* to page through output more easily.

Lessons Learned:
I learned that the proc.c file has important process and scheduling code, and that idle() is called when there are no runnable processes.

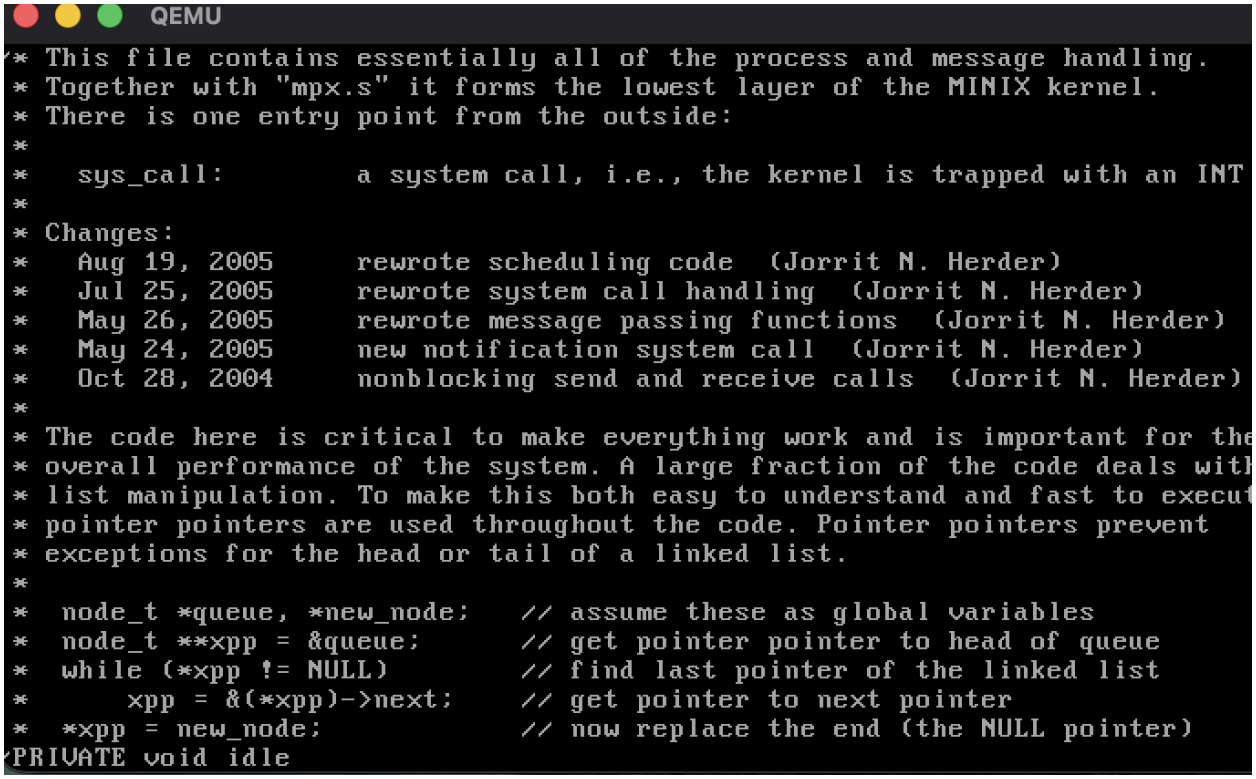
Step 3: Edit proc.c

Approach:
I opened proc.c in vi:

vi proc.c

Then I searched for the idle function inside vi with:

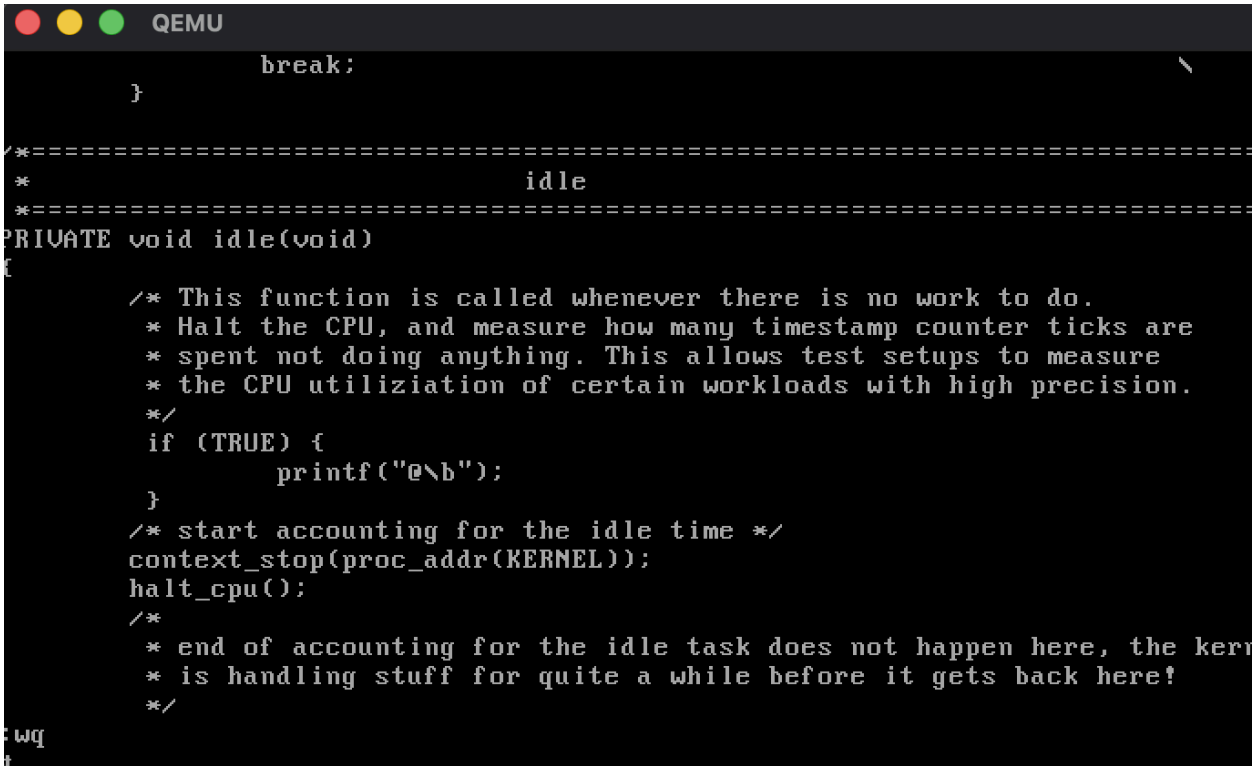
/PRIVATE void idle



Inside idle(), I added the following block before the idle accounting and CPU halt code:

```
if (TRUE) {
    printf("@\b");
}
```

The @ displays the symbol, and \b moves the cursor back over it so the cursor appears on top of the @.



Problems Encountered:
Editing in vi was difficult because I had to be careful not to change the other kernel code.

Solution:
After saving, I checked the function again to make sure the change was in the correct place.

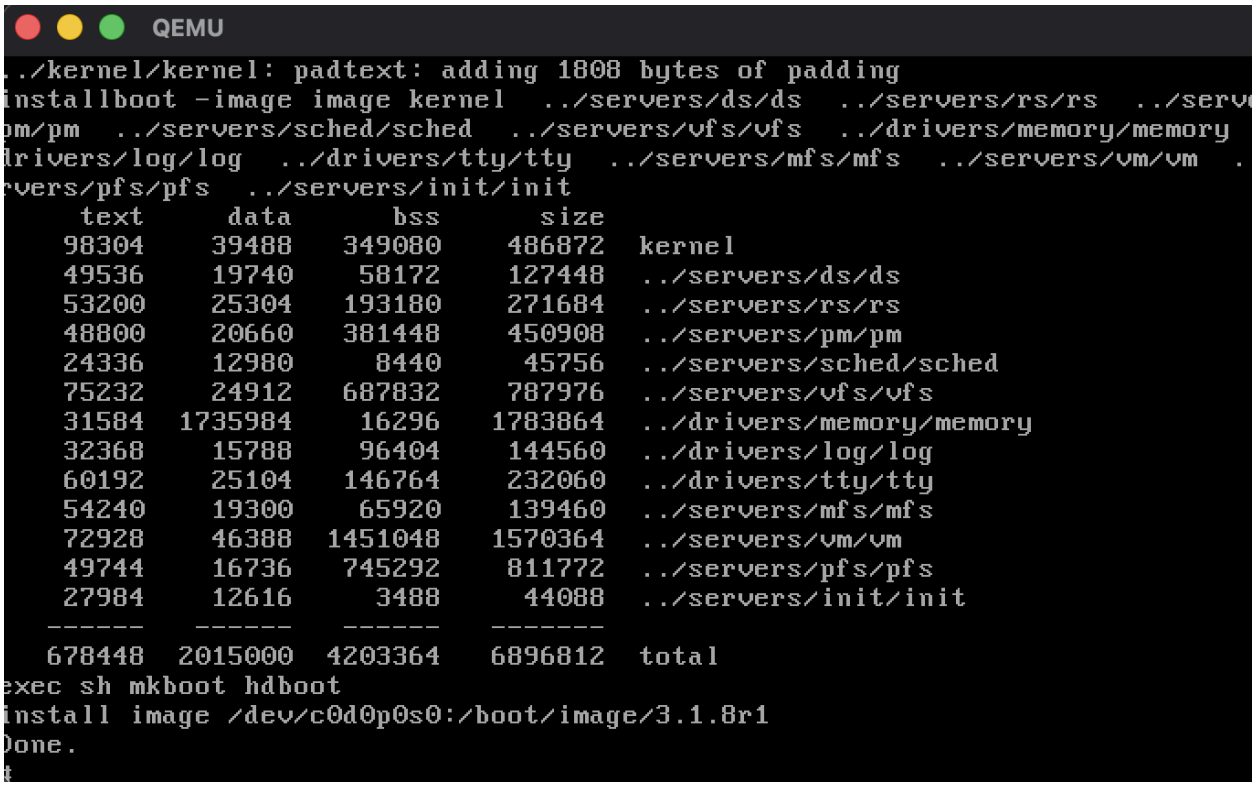
Lessons Learned:
I learned that context_stop() is used for accounting, while halt_cpu() is what actually stops the CPU until an interrupt occurs. Because of that, I placed the print statement before those lines.

Step 4: Rebuild and install the modified kernel

Approach:
After editing the source code, I rebuilt and installed the kernel image from /usr/src/tools:

cd /usr/src/tools
make hdboot

This compiled the kernel and installed the new boot image.



Problems Encountered:
N/A.

Solution:
N/A.

Lessons Learned:
I learned that make hdboot builds the image and installs it as a bootable MINIX image.

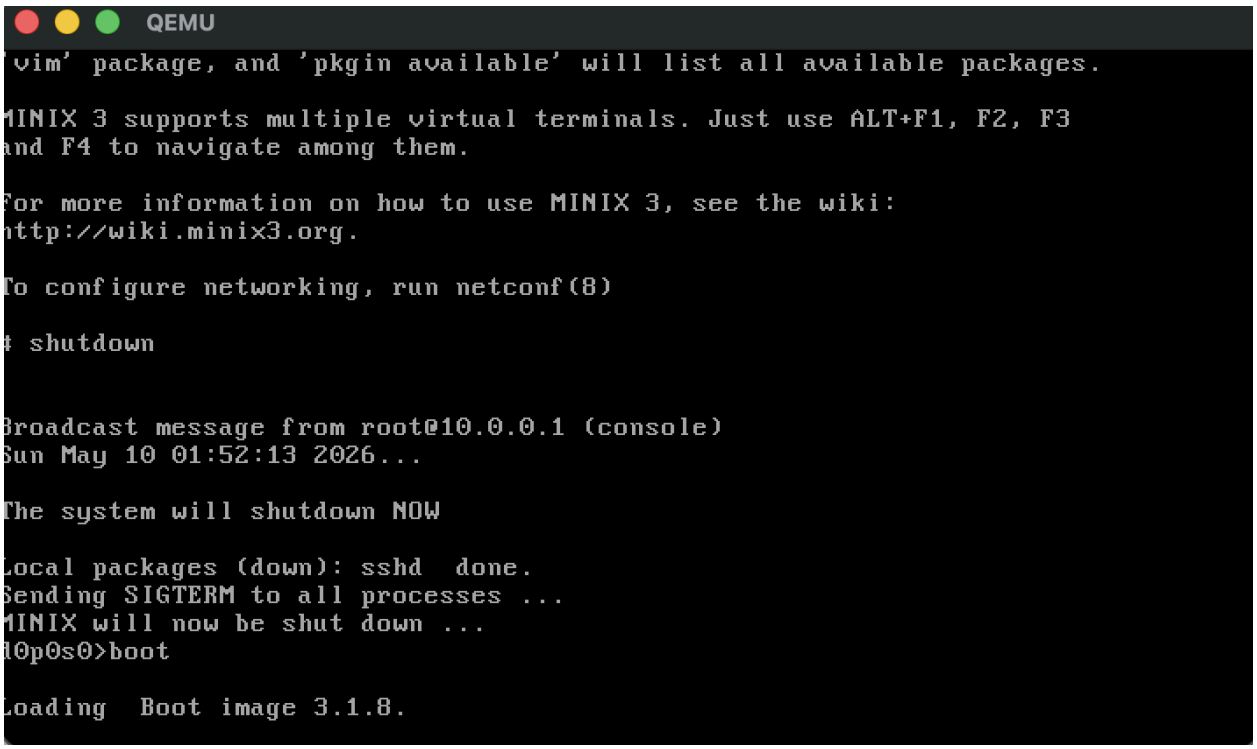
Step 5: Reboot into the modified kernel

Approach:
After *make hdboot* finished, I rebooted MINIX with:

shutdown -R now

This performed a hardware-style reset, which worked better in QEMU than manually using the boot monitor's boot command.

Problems Encountered:
When I used the boot monitor command boot, QEMU sometimes got stuck at Loading Boot image image.



*note: The time is inconsistent because I forgot to take a screenshot of the issue earlier.

Solution:
I used shutdown -R now for rebooting after the rebuild. When the boot monitor got stuck, I quit QEMU from the Mac terminal and restarted it.

Lessons Learned:
I learned that MINIX shutdown matters because it flushes filesystem changes safely, and that *shutdown -R now* was more reliable in my QEMU setup.

Task 2: Test it

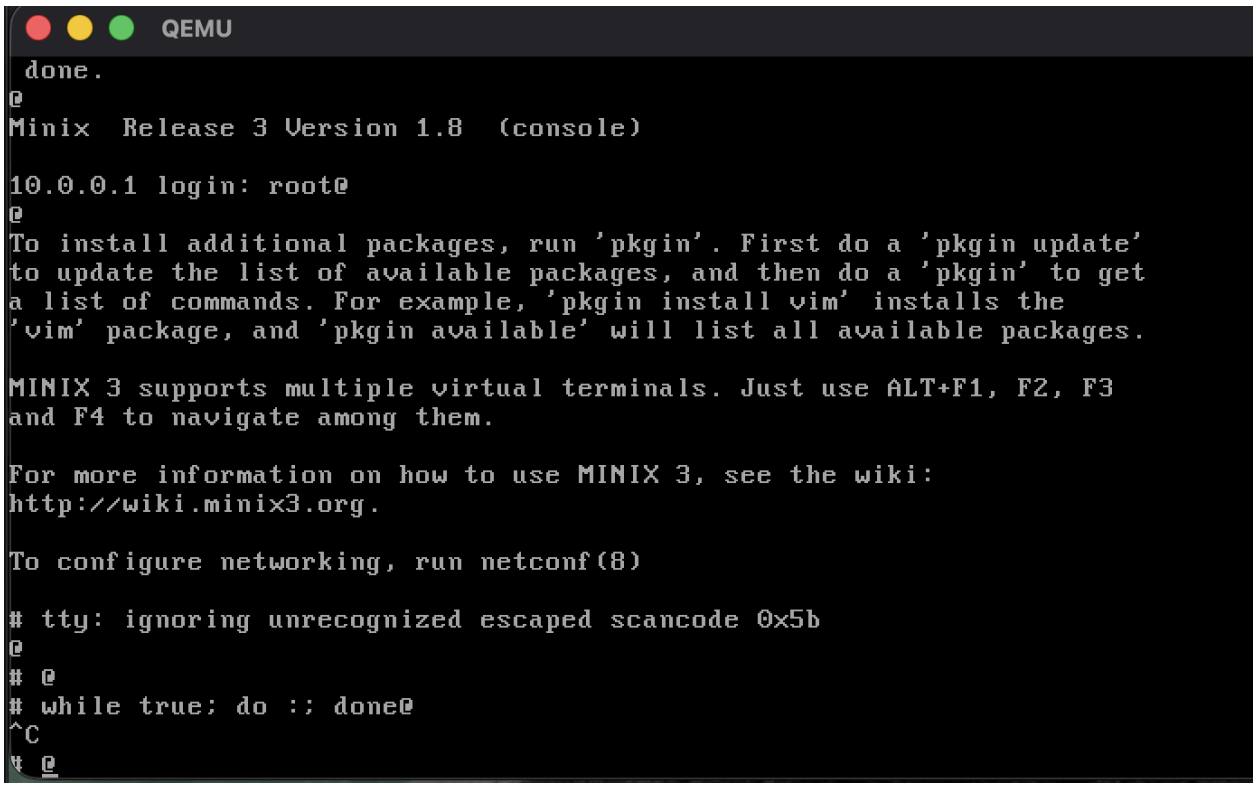
Step 6: Test the modification

Approach:
After booting the modified kernel, I logged in as root and saw the @ appear near the cursor when the system was idle. To test that the symbol only appeared when the system was idle, I ran a busy loop:

while true; do :; done

Then I stopped it with:

Ctrl+C



The loop kept the shell busy. While it was running, the @ stopped. After I pressed *Ctrl+C*, the system returned to idle and the @ appeared again.

Problems Encountered:
At first, I was not sure how to create a simple busy-waiting test in the MINIX shell. I needed a command that would keep the CPU busy without printing a lot of output to the screen, so I looked up how shell infinite loops and no-op commands work.

Solution:
I used the busy loop:

while true; do :; done

I learned that true can be used as a command that always succeeds, so while true creates an infinite loop. I also learned that : is the shell null utility, meaning it does nothing but still counts

as a valid command inside the loop. This let me keep the shell busy without producing output. I stopped the loop with *Ctrl+C*.

References:
POSIX : null utility: https://www.unix.com/man-page/posix/1p/colon
ShellCheck explanation of empty loops/no-op commands:
https://www.shellcheck.net/wiki/SC1060
true command: https://linuxcommandlibrary.com/man/true

Lessons Learned:
I learned that a busy-waiting loop can be created by repeatedly running a command that does nothing. This gave me a clear comparison between a busy system and an idle system. When the busy loop was running, the @ output stopped, and when I stopped the loop, the @ returned.

Step 7: Return to the old kernel

Approach:
After testing, I wanted to stop using the modified kernel because the @ output was distracting. I shut down to the boot monitor:

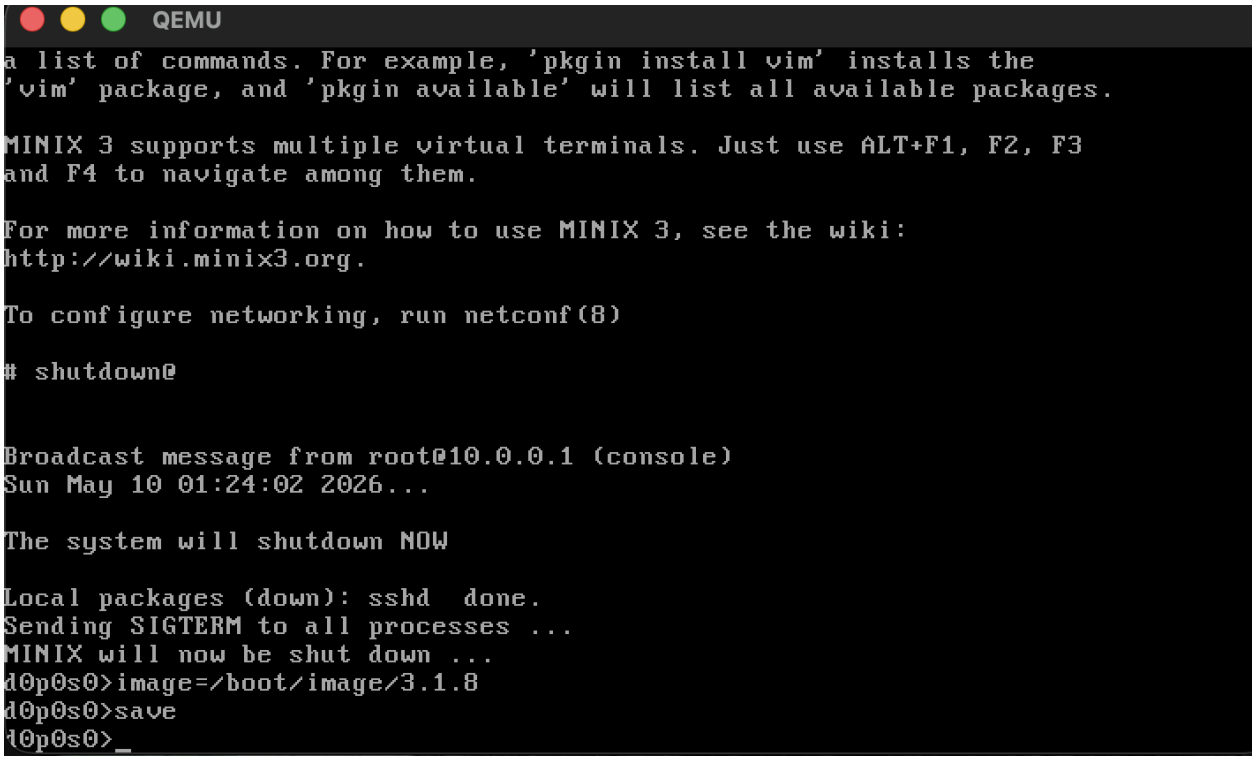
shutdown

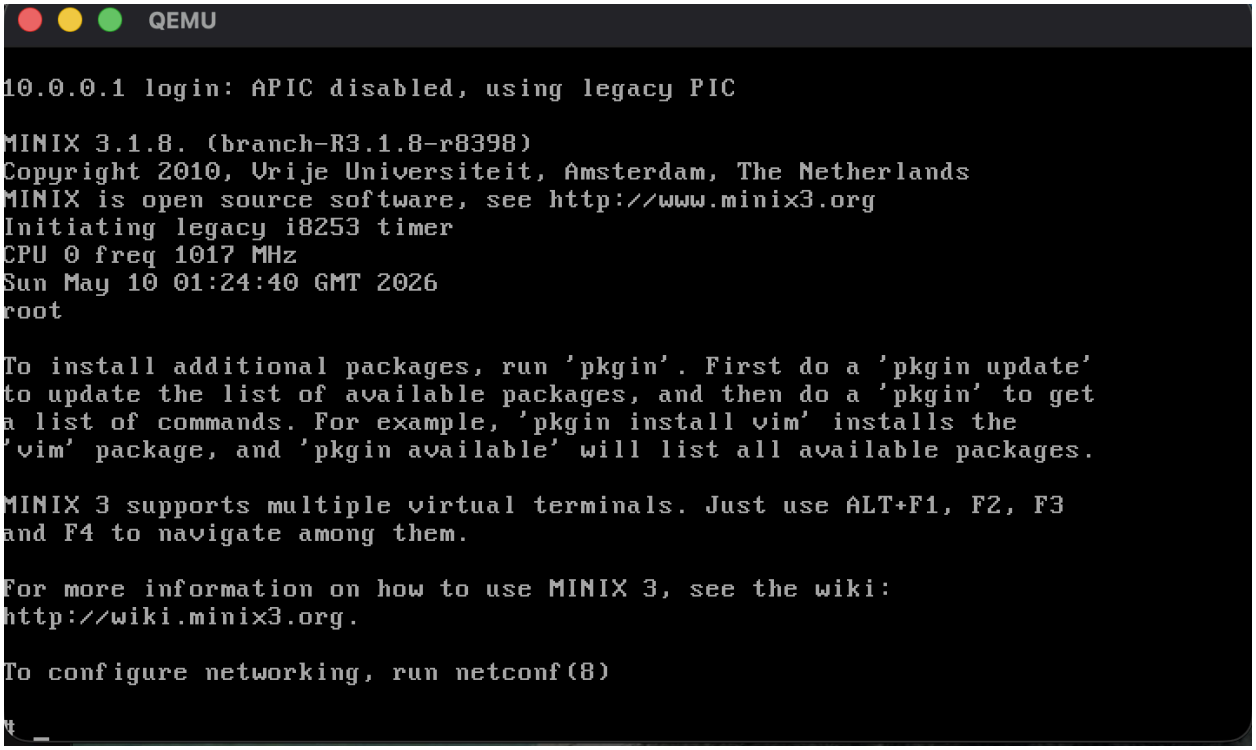
Then I listed the available boot images:

ls /boot/image

I selected the old stable image and saved it as the default:

image=/boot/image/3.1.8
save





Problems Encountered:
N/A.

Solution:
N/A.

Lessons Learned:
I learned that MINIX keeps multiple boot images, and the boot monitor can choose which one to boot. This is useful when a modified kernel works but should not stay as the default.